Predicting Workforce Burnout Risk Using Economic and Public Health Indicators

Allyson Totpal

DSC680

March 8, 2026

Business Problem

Employee burnout has become a major operational and financial risk for organizations, leading to reduced productivity, higher turnover, absenteeism, and increased healthcare costs. However, burnout is often measured reactively through surveys rather than predicted proactively. The core business problem this project seeks to address is whether workforce burnout risk can be predicted using publicly available economic, labor market, and mental health indicators. The project may also explore macroeconomic and labor indicators that may strongly be associated with burnout-related outcomes.

Background/History

Economic cycles have long been associated with mental health outcomes. Research following the Great Recession demonstrated links between unemployment and depression, while pandemic-era labor market disruptions intensified attention on psychological distress. Despite national discussions around burnout, limited empirical work integrates state unemployment dynamics, real personal income trends, national labor signals, behavior sentiment indicators, and state-level mental health measures. This study constructs a quarterly state-level panel to quantify these relationships systematically.

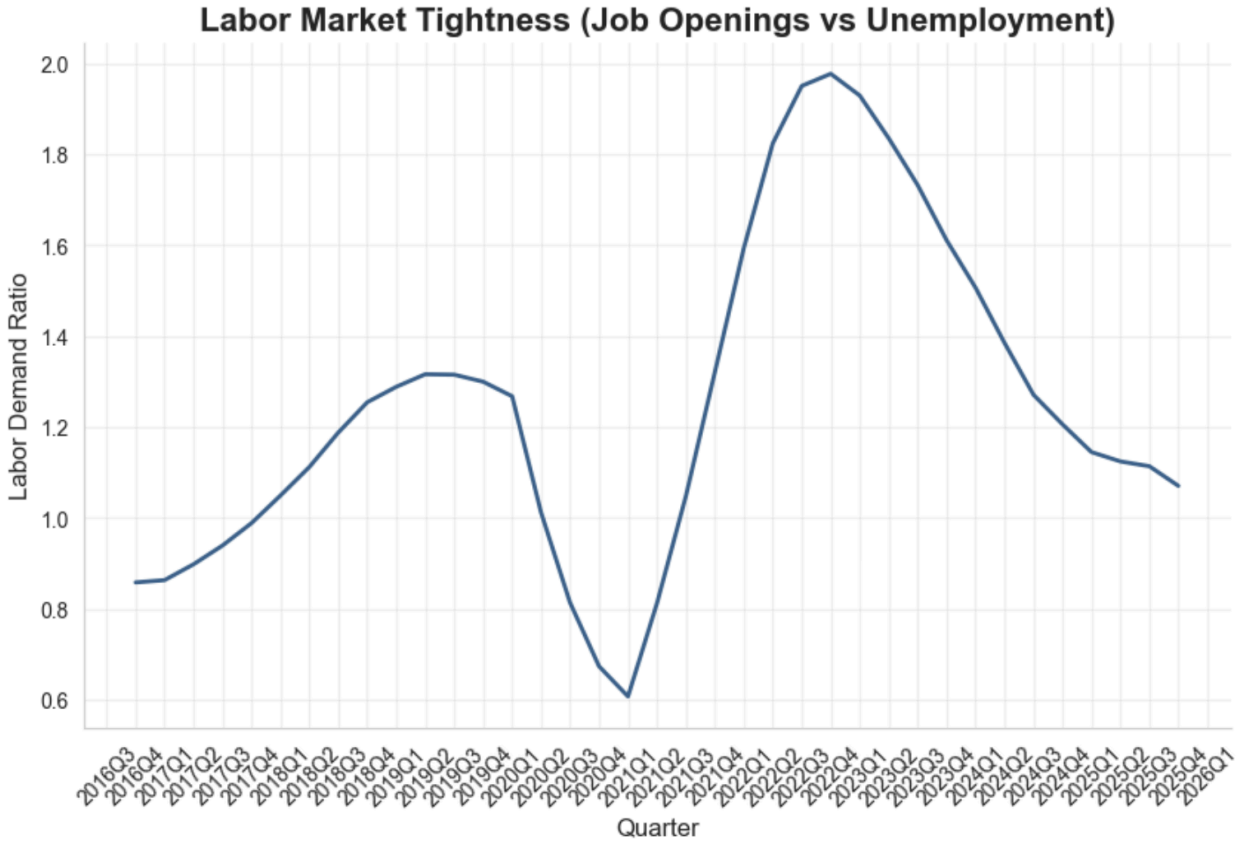
Data Explanation

The project will integrate multiple publicly available datasets at the state, and where possible, industry level. Labor market indicators will be sourced from the U.S. Bureau of Labor Statistics (BLS), including unemployment rates, job openings data from Job Openings and Labor Turnover Survey (JOLTS), quit rates, wage growth, average weekly hours worked, and industry-level employment statistics. Mental health indicators will be drawn from the CDC Household Pulse Survey (HPS), including measures of anxiety symptoms, depressive symptoms,

and reported difficulty concentrating, which will serve as proxy indicators of psychological strain. If necessary, supplemental data from the Behavioral Risk Factor Surveillance System (BRFSS) may be incorporated to capture frequent mental distress days. Economic context variables such as income growth and inflation-adjusted earnings will be obtained from the Bureau of Economic Analysis (BEA). Optionally, Google Trends search interest for terms such as “burnout” and “job stress” may be used as a behavioral proxy signal. These datasets will be aggregated to a common time frequency and merged by state and time period to construct a panel dataset suitable for predictive modeling.

All datasets were standardized to a quarterly time scale and merged into a single analytical dataset. Data cleaning included removing missing values, aligning time periods across datasets, and converting economic indicators to consistent numeric formats.

Several engineered variables were created to better capture economic pressure.



These included a Labor Market Stress Index derived from standardized labor indicators, a real wage growth measure calculated as wage growth minus inflation, and a labor demand ratio representing the relationship between job openings and unemployment. Lagged variables and change indicators were also constructed to capture delayed effects of economic conditions on burnout signals.

After preprocessing, the final dataset contained quarterly observations with engineered features designed to reflect both economic conditions and behavioral signals related to workforce stress.

Data Dictionary

Field name	Description	Source	Type
`quarter`	YYYYQ#	Combined dataset	string
`unemployment_rate`	National unemployment rate	BLS	float
`weekly_hours`	Average weekly hours worked	BLS	float
`hourly_earnings`	Average hourly earnings	BLS	float
`job_opening_rates`	Rate of job openings relative to total employment	BLS	float
`bea_personal_income`	Aggregate personal income reflecting overall economic conditions	BEA	float

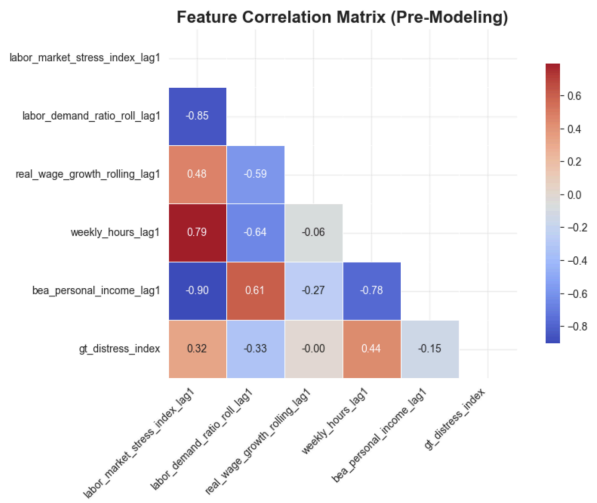
hps_anxiety_pct	Percentage of survey respondents reporting anxiety symptoms	HPS	float
hps_depression_pct	Percentage of survey respondents reporting depressive symptoms	HPS	float
trends_distress_index	Composite search intensity index	Google Trends	float
labor_market_stress_index	Standardized composite index measuring labor market pressure	Engineered variable	float
real_wage_growth	Wage growth adjusted for inflation, measuring real purchasing power change	Engineered variable	float
labor_demand_ratio	Ratio of job openings to unemployment	Engineered variable	float
stress_change_lag1	Lagged quarterly change in the labor market stress index	Engineered variable	float
wage_change_lag1	Lagged quarterly change in real wage growth	Engineered variable	float
demand_change_lag1	Lagged quarterly change in labor demand ratio	Engineered variable	float
hours_change_lag1	Lagged quarterly change in average weekly hours worked	Engineered variable	float
time_index	Sequential index representing time progression across quarters	Engineered variable	int
quarter_num	Numeric representation of quarter (1-4) capturing seasonal patterns	Engineered variable	int

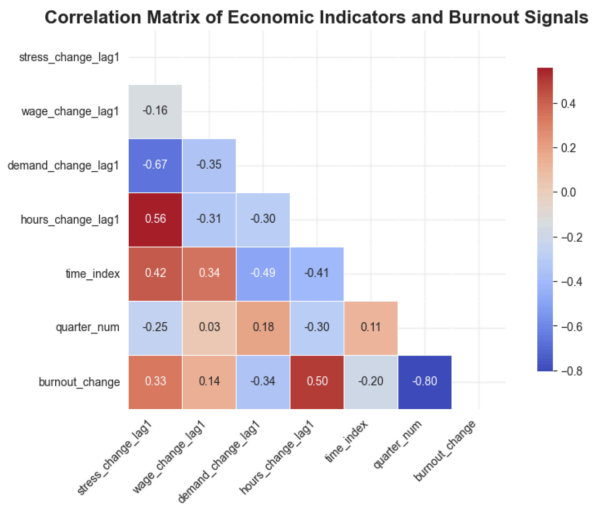
Methods

This study used a predictive analytics approach to examine whether economic and public health indicators could forecast signals of workforce burnout. After cleaning and merging datasets from multiple sources, the data were standardized to a quarterly time scale and several engineered variables were created to capture labor market pressure. These included a Labor Market Stress Index constructed from standardized unemployment, job openings, weekly hours worked, and quit rate indicators, as well as measures of real wage growth and labor demand

ratios. Lagged variables and change indicators were generated to model delayed effects of economic conditions on burnout-related behavioral signals. Three predictive models were trained to evaluate these relationships: linear regression as a baseline statistical model, Random Forest regression to capture nonlinear relationships between variables, and XGBoost as a gradient boosting method designed for high predictive performance. Model performance was evaluated using root mean squared error (RMSE) and the coefficient of determination (R^2), and a time-series cross-validation was applied to reduce instability caused by the limited number of observations.

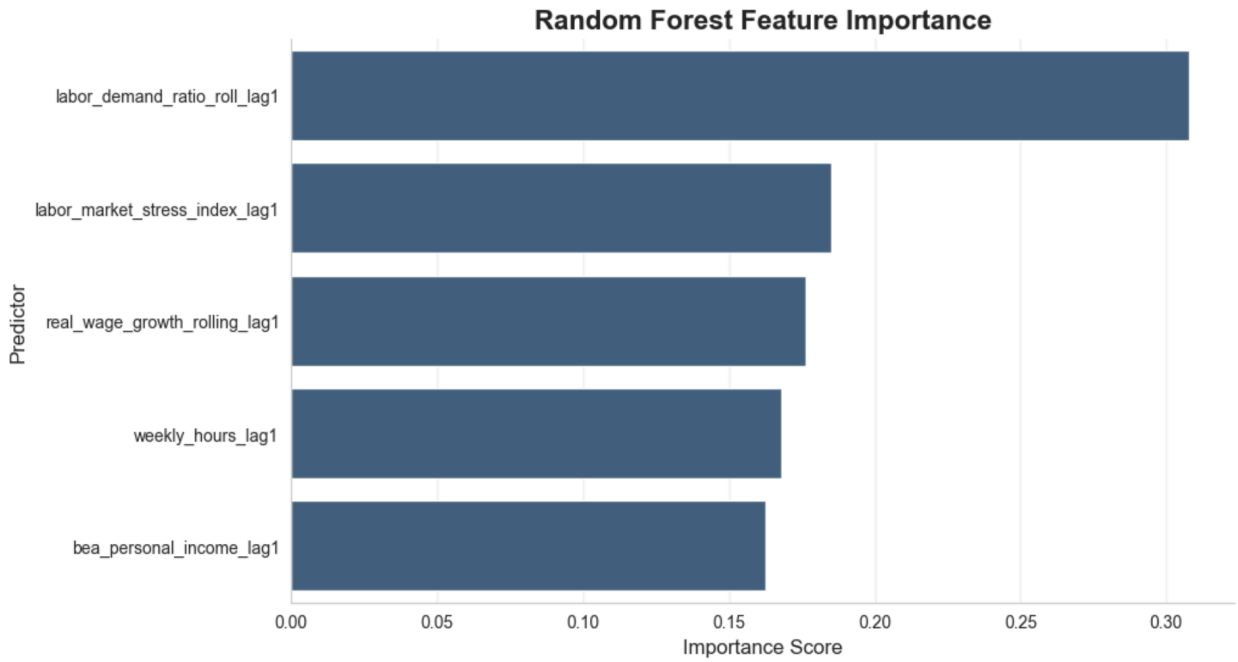
Analysis





The modeling results indicate that economic pressure indicators provide measurable predictive insight into workforce burnout signals. Among the models tested, Random Forest produced the lowest prediction error, suggesting that nonlinear relationships exist between economic variables and burnout-related search behavior. Linear regression performed less effectively, likely due to the limited dataset size and the complex interactions between labor

market indicators.



Feature importance analysis revealed that variables representing changes in economic conditions were particularly influential predictors, including changes in the Labor Market Stress Index, real wage growth, and labor demand ratios. These findings suggest that workers may respond more strongly to sudden shifts in labor market conditions rather than stable economic levels. Although predictive performance was constrained by the relatively small number of observations available after aligning all datasets to quarterly intervals, the results provide evidence that macroeconomic labor conditions are associated with behavioral signals related to workplace stress and burnout.

Assumptions, Limitations, & Challenges

Assumptions made are quarterly aggregation sufficiently captures underlying economic stress dynamics, HPS self-reported symptom percentages approximate population-level distress, Google Trends search intensity reflects national psychological salience, and state fixed effects absorb unobserved time-invariant heterogeneity. Due to the nature of data collected, some limitations are that Google Trends is national, not state-specific data, HPS survey methodology

has changed across waves, income is reported in millions and population normalization not yet applied, and causal inference cannot be fully established without exogenous instruments.

Future Uses, Implementation, & Ethical Assessment

The modeling framework developed in this study could support future workforce analytics and early-warning monitoring systems designed to detect rising workforce stress and burnout risk. By integrating economic indicators with behavioral signals such as search trends, organizations and policymakers could monitor labor market conditions and identify periods of elevated workforce pressure, enabling earlier interventions such as workload adjustments, expanded mental health resources, or policy responses aimed at improving worker well-being. Because many of the data sources used in this study are regularly updated, the analytical pipeline could be automated to produce ongoing monitoring dashboards for workforce conditions. However, ethical considerations must guide the use of such models. Although the data used in this study are aggregated and publicly available, predictive insights about workforce stress should not be used to make decisions that unfairly impact employees or specific labor groups.

Appendix

```
import os
import re
import json
import requests
import numpy as np
import pandas as pd
# define analysis year range
start_year = 2016
end_year = 2026

bea_api_url = 'https://apps.bea.gov/api/data'
bea_api_key = os.getenv('bls_api_key', 'E40FE19C-D71C-4C44-AB85-6D7CEA7B7051')

# load hps and google trends files
hps =
pd.read_csv('/Users/smooshii/DSC680/Indicators_of_Anxiety_or_Depression_Based_on_Repor
ted_Frequency_of_Symptoms_During_Last_7_Days_20260301.csv')
trends =
pd.read_csv('/Users/smooshii/DSC680/time_series_US_20160101-0000_20260301-1159.csv')
# state helpers
state_abbr_to_name = {'AL': 'Alabama', 'AK': 'Alaska', 'AZ': 'Arizona', 'AR':
'Arkansas', 'CA': 'California',
                      'CO': 'Colorado', 'CT': 'Connecticut', 'DE': 'Delaware', 'DC':
'District of Columbia', 'FL': 'Florida',
                      'GA': 'Georgia', 'HI': 'Hawaii', 'ID': 'Idaho', 'IL':
'Illinois', 'IN': 'Indiana', 'IA': 'Iowa', 'KS': 'Kansas',
                      'KY': 'Kentucky', 'LA': 'Louisiana', 'ME': 'Maine', 'MD':
'Maryland', 'MA': 'Massachusetts', 'MI': 'Michigan',
                      'MN': 'Minnesota', 'MS': 'Mississippi', 'MO': 'Missouri', 'MT':
'Montana', 'NE': 'Nebraska', 'NV': 'Nevada',
                      'NH': 'New Hampshire', 'NJ': 'New Jersey', 'NM': 'New Mexico',
'NY': 'New York', 'NC': 'North Carolina',
                      'ND': 'North Dakota', 'OH': 'Ohio', 'OK': 'Oklahoma', 'OR':
'Oregon', 'PA': 'Pennsylvania', 'RI': 'Rhode Island',
                      'SC': 'South Carolina', 'SD': 'South Dakota', 'TN': 'Tennessee',
'TX': 'Texas', 'UT': 'Utah', 'VT': 'Vermont',
                      'VA': 'Virginia', 'WA': 'Washington', 'WV': 'West Virginia',
'WI': 'Wisconsin', 'WY': 'Wyoming'}
# reverse mapping from full state names to abbreviations
state_name_to_abbr = {v: k for k, v in state_abbr_to_name.items()}
# hps clean
hps = hps[hps['Group'].eq('By State')].copy()

# parse date and create quarter
hps['date'] = pd.to_datetime(hps['Time Period End Date'], errors = 'coerce')
hps = hps[hps['date'].notna()].copy()
hps['quarter'] = hps['date'].dt.to_period('Q').astype(str)

# map state names to abbreviations
hps['state'] = hps['State'].astype(str).str.strip().map(state_name_to_abbr)
hps = hps[hps['state'].isin(state_abbr_to_name.keys())].copy()

# keep only anxiety + depression
keep_inds = ['Symptoms of Anxiety Disorder', 'Symptoms of Depressive Disorder']
hps = hps[hps['Indicator'].isin(keep_inds)].copy()

# ensure numeric
hps['Value'] = pd.to_numeric(hps['Value'], errors = 'coerce')

# pivot to columns and aggregate quarterly mean (percent)
```

```
hps_q = (hps.pivot_table(index = ['state', 'quarter'],
                         columns = 'Indicator',
                         values = 'Value',
                         aggfunc = 'mean').reset_index().rename(columns = {'Symptoms
of Anxiety Disorder': 'hps_anxiety_pct',
                                                                            'Symptoms
of Depressive Disorder': 'hps_depression_pct'})
        )

# filter years
hps_q['year'] = hps_q['quarter'].str.slice(0, 4).astype(int)
hps_q = hps_q[(hps_q['year'] >= start_year) & (hps_q['year'] <=
end_year)].drop(columns = ['year'])

hps_q.head()
# google trends clean
trends['date'] = pd.to_datetime(trends['Time'], errors = 'coerce')
trends = trends[trends['date'].notna()].copy()

term_cols = [c for c in trends.columns if c not in ['Time', 'date']]
for c in term_cols:
    trends[c] = (trends[c].astype(str)
                 .str.replace('<1', '0.5', regex = 'False')
                 .str.strip())
    trends[c] = pd.to_numeric(trends[c], errors = 'coerce')

# drop any completely non-numeric columns
term_cols = [c for c in term_cols if pd.api.types.is_numeric_dtype(trends[c])]

# convert to quarter
trends['quarter'] = trends['date'].dt.to_period('Q').astype(str)

# aggregate monthly to quarterly
trends_q = trends.groupby('quarter', as_index = False)[term_cols].mean()
# composite distress index
trends_q['gt_distress_index'] = trends_q[term_cols].mean(axis = 1)

# filter years
trends_q['year'] = trends_q['quarter'].str.slice(0, 4).astype(int)
trends_q = trends_q[(trends_q['year'] >= start_year) & (trends_q['year'] <=
end_year)].drop(columns = ['year'])

trends_q.head()
# if no api key available, create empty fallback dataframe
if not bea_api_key:
    print('[NOTE] BEA_API_KEY is not set.')
    bea_income = pd.DataFrame(columns = ['state', 'quarter', 'bea_personal_income'])

else:
    # get valid regional table names
    tbl_params = {'UserID': bea_api_key,
                  'method': 'GetParameterValues',
                  'datasetname': 'Regional',
                  'ParameterName': 'TableName',
                  'ResultFormat': 'JSON'}

    tbl_resp = requests.get('https://apps.bea.gov/api/data', params = tbl_params,
timeout = 60)
    tbl_resp.raise_for_status()
    tbl_json = tbl_resp.json()

    # convert table metadata to dataframe
    table_rows = pd.DataFrame(tbl_json['BEAAPI']['Results']['ParamValue'])
```

```
    # detect likely column names for table code and description
    tname_col = [c for c in table_rows.columns if c.lower() in ['key', 'tablename',
'paramvalue']]
    tdesc_col = [c for c in table_rows.columns if c.lower() in ['desc',
'description']]

    tname_col = tname_col[0] if tname_col else table_rows.columns[0]
    tdesc_col = tdesc_col[0] if tdesc_col else None

    # keep likely quarterly state income tables
    candidate_tables = table_rows.copy()
    candidate_tables['table_name'] = candidate_tables[tname_col].astype(str)

    if tdesc_col is not None:
        candidate_tables['table_desc'] = candidate_tables[tdesc_col].astype(str)
    else:
        candidate_tables['table_desc'] = ''

    candidate_tables = candidate_tables[
        candidate_tables['table_name'].str.contains(r'^SQINC', case=False, na=False) |
        candidate_tables['table_desc'].str.contains('quarter', case=False, na=False) &
        candidate_tables['table_desc'].str.contains('income', case=False, na=False) &
        candidate_tables['table_desc'].str.contains('state', case=False,
na=False)].copy()

    # prefer SQINC4 if available, otherwise use first candidate
    if (candidate_tables['table_name'].str.upper() == 'SQINC4').any():
        quarterly_table = 'SQINC4'
    else:
        quarterly_table = candidate_tables['table_name'].iloc[0]

    # get valid linecode values for selected table
    line_params = {'UserID': bea_api_key,
                   'method': 'GetParameterValuesFiltered',
                   'datasetname': 'Regional',
                   'TargetParameter': 'LineCode',
                   'TableName': quarterly_table,
                   'GeoFips': 'STATE',
                   'ResultFormat': 'JSON'}

    line_resp = requests.get('https://apps.bea.gov/api/data', params = line_params,
timeout = 60)
    line_resp.raise_for_status()
    line_json = line_resp.json()

    # convert line metadata to dataframe
    line_rows = pd.DataFrame(line_json['BEAAPI']['Results']['ParamValue'])

    # detect likely line code and description columns
    lcode_col = [c for c in line_rows.columns if c.lower() in ['key', 'linecode',
'paramvalue']]
    ldesc_col = [c for c in line_rows.columns if c.lower() in ['desc', 'description']]

    lcode_col = lcode_col[0] if lcode_col else line_rows.columns[0]
    ldesc_col = ldesc_col[0] if ldesc_col else None

    line_rows['line_code'] = line_rows[lcode_col].astype(str)
    if ldesc_col is not None:
        line_rows['line_desc'] = line_rows[ldesc_col].astype(str)
    else:
        line_rows['line_desc'] = ''
```

```
    # try to auto-find personal income automatically
    personal_income_match = line_rows[
        line_rows['line_desc'].str.contains(r'\bpersonal income\b', case=False,
na=False)
    ].copy()

    if len(personal_income_match) == 0:
        raise ValueError(
            "Could not auto-find the personal income line code. "
            "Check the printed 'Available line codes' output and set target_line_code
manually."
        )

    target_line_code = personal_income_match['line_code'].iloc[0]
    print('\nUsing line code:', target_line_code)
    print('Description:', personal_income_match['line_desc'].iloc[0])

    # pull quarterly data
    data_params = {'UserID': bea_api_key,
                   'method': 'GetData',
                   'datasetname': 'Regional',
                   'TableName': quarterly_table,
                   'LineCode': target_line_code,
                   'GeoFIPS': 'STATE',
                   'Year': 'ALL',
                   'Frequency': 'Q',
                   'ResultFormat': 'JSON'}

    data_resp = requests.get('https://apps.bea.gov/api/data', params = data_params,
timeout = 60)
    data_resp.raise_for_status()
    data_json = data_resp.json()

    # handles bea api errors
    if 'Error' in data_json.get('BEAAPI', {}):
        print('BEA API ERROR:')
        print(data_json['BEAAPI']['Error'])
        bea_income = pd.DataFrame(columns=['state', 'quarter', 'bea_personal_income'])
    else:
        # convert bea results to dataframe
        bea = pd.DataFrame(data_json['BEAAPI']['Results']['Data'])

        # numeric cleanup
        bea['DataValue'] = (
            bea['DataValue']
            .astype(str)
            .str.replace(',', '', regex=False)
            .replace('(NA)', np.nan)
        )
        bea['DataValue'] = pd.to_numeric(bea['DataValue'], errors = 'coerce')

        #clean geography names
        bea['GeoName_clean'] = (bea['GeoName'].astype(str)
                                .str.strip()
                                .str.replace(r'\s*\*$', '', regex=True)
                                .str.replace(r'\s+', ' ', regex=True)
                               )

        # map state names to abbreviations
        bea['state'] = bea['GeoName_clean'].map(state_name_to_abbr)

        # keep only state rows
        bea = bea[bea['state'].notna()].copy()
```

```
        # keep only quarterly periods like 2016Q1
        bea['quarter'] =
bea['TimePeriod'].astype(str).str.extract(r'(\d{4}Q[1-4])')[0]
        bea = bea[bea['quarter'].notna()].copy()

        # filter years
        bea['year'] = pd.to_numeric(bea['quarter'].str[:4], errors = 'coerce')
        bea = bea[(bea['year'] >= start_year) & (bea['year'] <= end_year)].copy()

        # final output
        bea_income = (bea[['state', 'quarter', 'DataValue']]
                      .rename(columns = {'DataValue': 'bea_personal_income'})
                      .sort_values(['state', 'quarter'])
                      .reset_index(drop = True)
                     )

bea_income.head()
file_path = '/Users/smooshii/DSC680/'

def clean_bls_series(file_path, value_name):
    # read file without headers first
    raw = pd.read_excel(file_path, header = None)

    # find the row where the actual table starts
    start_row = raw[raw[0] == 'Year'].index[0]

    # read again using that row as header
    df = pd.read_excel(file_path, skiprows=start_row)

    # rename first column
    df = df.rename(columns = {'Year': 'year'})

    # keep only actual month columns
    valid_months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

    keep_cols = ['year'] + [col for col in df.columns if col in valid_months]
    df = df[keep_cols]

    # reshape wide to long
    df = df.melt(id_vars='year',
                 var_name='month',
                 value_name=value_name
                )

    # drop missing values
    df = df.dropna(subset = [value_name])

    # make sure year is numeric
    df['year'] = pd.to_numeric(df['year'], errors = 'coerce')
    df = df.dropna(subset = ['year'])
    df['year'] = df['year'].astype(int)

    # create date
    df['date'] = pd.to_datetime(df['year'].astype(str) + '-' + df['month'],
                                format = '%Y-%b', errors = 'coerce')

    # remove anything that still failed parsing
    df = df.dropna(subset = ['date'])

    # sort and return
    df = df[['date', value_name]].sort_values('date').reset_index(drop = True)
```

```
    return df
# load and clean each bls series
weekly_hours = clean_bls_series('SeriesReport-20260307143233_430752.xlsx',
'weekly_hours')
hourly_earnings = clean_bls_series('SeriesReport-20260307143243_65a02b.xlsx',
'hourly_earnings')
cpi = clean_bls_series('SeriesReport-20260307143031_390ce4.xlsx', 'cpi')
employment_level = clean_bls_series('SeriesReport-20260307143214_5b275f.xlsx',
'employment_level')
unemployment_rate = clean_bls_series('SeriesReport-20260307143005_720938.xlsx',
'unemployment_rate')
quit_rate = clean_bls_series('SeriesReport-20260307143345_88d729.xlsx', 'quit_rate')
job_openings = clean_bls_series('SeriesReport-20260307143341_05f6ea.xlsx',
'job_openings_rate')
# merge all bls series into single monthly dataframe
bls_data = (
    weekly_hours
    .merge(hourly_earnings, on='date', how='outer')
    .merge(cpi, on='date', how='outer')
    .merge(employment_level, on='date', how='outer')
    .merge(unemployment_rate, on='date', how='outer')
    .merge(quit_rate, on='date', how='outer')
    .merge(job_openings, on='date', how='outer')
    .sort_values('date')
    .reset_index(drop=True)
)

bls_data.head()
# aggregate bls monthly data to quarterly

# ensure date column is datetime
bls_data['date'] = pd.to_datetime(bls_data['date'])

# keep only 2014 through 2026
bls_data = bls_data[
    (bls_data['date'].dt.year >= 2014) &
    (bls_data['date'].dt.year <= 2026)].copy()

# create quarter variable
bls_data['quarter'] = bls_data['date'].dt.to_period('Q')

# aggregate monthly data to quarterly averages
bls_quarterly = (bls_data.groupby('quarter', as_index = False)
                 .agg({'weekly_hours': 'mean',
                       'hourly_earnings': 'mean',
                       'cpi': 'mean',
                       'employment_level': 'mean',
                       'unemployment_rate': 'mean',
                       'quit_rate': 'mean',
                       'job_openings_rate': 'mean'
                      })
                )

# convert quarter to timestamp if you want an actual date column
bls_quarterly['quarter_start'] = bls_quarterly['quarter'].dt.start_time
bls_quarterly['year'] = bls_quarterly['quarter'].dt.year
bls_quarterly['qtr'] = bls_quarterly['quarter'].dt.quarter

# reorder columns
bls_quarterly = bls_quarterly[['quarter', 'quarter_start', 'year', 'qtr',
'weekly_hours',
```

```
                                'hourly_earnings', 'cpi', 'employment_level',
'unemployment_rate',
                                'quit_rate',
'job_openings_rate']].sort_values('quarter_start').reset_index(drop = True)
# convert quarter period to string
bls_quarterly['quarter'] = bls_quarterly['quarter'].astype(str)
# keep only quarters from 2016 onward
bls_quarterly = bls_quarterly[bls_quarterly['year'] >= 2016].reset_index(drop = True)
bls_quarterly.head()
# average state-level bea income to national quarterly mean
bea_nat = (bea_income.groupby('quarter', as_index = False)
           .agg({'bea_personal_income':'mean'}))


hps_nat = (hps_q.groupby('quarter', as_index=False)
           .mean(numeric_only=True))
# merging
# start with quarterly bls data
master_df = bls_quarterly.copy()
# merge bea income
master_df = master_df.merge(bea_nat, on = 'quarter', how = 'left')
# merge hps
master_df = master_df.merge(hps_nat, on = 'quarter', how = 'left')
# merge google trends
master_df = master_df.merge(trends_q, on = 'quarter', how = 'left')

# sort final master dataset chronologically
master_df = master_df.sort_values('quarter').reset_index(drop=True)
master_df.head()
# create lagged macroeconomic predictors
lag_vars = ['unemployment_rate', 'quit_rate', 'weekly_hours', 'bea_personal_income']

# create 1-quarter lag for each variable
for col in lag_vars:
    master_df[col + '_lag1'] = master_df[col].shift(1)
from sklearn.preprocessing import StandardScaler

stress_vars = ['unemployment_rate', 'quit_rate', 'weekly_hours', 'job_openings_rate']
scaler = StandardScaler()

# standarize stress variables
master_df[[v + '_z' for v in stress_vars]] =
scaler.fit_transform(master_df[stress_vars])

# invert job openings
master_df['job_openings_rate_z'] = -master_df['job_openings_rate_z']
# average standardized components into composite labor market stress index
master_df['labor_market_stress_index'] = master_df[['unemployment_rate_z',
                                                    'quit_rate_z',
                                                    'weekly_hours_z',

'job_openings_rate_z']].mean(axis=1)
# visualize labor market stress index
import matplotlib.pyplot as plt
import seaborn as sns

# style settings
sns.set_style('whitegrid')
plt.rcParams.update({'figure.figsize': (10,6),
                     'axes.titlesize': 16,
                     'axes.labelsize': 12,
                     'axes.titleweight': 'bold',
                     'font.family': 'sans-serif',
```

```
                     'grid.alpha': 0.3,
                     'lines.linewidth': 2,})
# custom color palette for plots
palette = {'stress': '#D1495B',
           'burnout': '#00798C',
           'prediction': '#EDA63B',
           'economic': '#30638E'}

plt.figure()
plt.plot(master_df['quarter'],
         master_df['labor_market_stress_index'],
         color = palette["stress"])
plt.title('Labor Market Stress Index Over Time')
plt.xlabel('Quarter')
plt.ylabel('Standardized Stress Index')
plt.xticks(rotation = 45)

sns.despine()
plt.show()
# nominal wage growth
master_df['wage_growth'] = master_df['hourly_earnings'].pct_change()
# inflation rate based on cpi growth
master_df['inflation_rate'] = master_df['cpi'].pct_change()
# real wage growth = wage growth minus inflation
master_df['real_wage_growth'] = (master_df['wage_growth'] -
master_df['inflation_rate'])
# smooth using 4-quarter rolling mean
master_df['real_wage_growth_rolling'] = (master_df['real_wage_growth']
                                         .rolling(window = 4)
                                         .mean())
# plot
plt.figure()
plt.plot(master_df['quarter'],
         master_df['real_wage_growth_rolling'],
         color = palette["economic"]
)
plt.title('Real Wage Growth (Inflation Adjusted)')
plt.xlabel('Quarter')
plt.ylabel('Growth Rate')
plt.xticks(rotation = 45)

sns.despine()
plt.show()
# ratio of job openings to unemployment
master_df['labor_demand_ratio'] = master_df['job_openings_rate'] /
master_df['unemployment_rate']
# standardized labor demand ratio
master_df['labor_demand_ratio_z'] =
scaler.fit_transform(master_df[['labor_demand_ratio']])
# smoothed labor demand ratio
master_df['labor_demand_ratio_roll'] = master_df['labor_demand_ratio'].rolling(window
= 4).mean()
plt.figure()
plt.plot(master_df['quarter'],
         master_df['labor_demand_ratio_roll'],
         color = palette["economic"])
plt.title('Labor Market Tightness (Job Openings vs Unemployment)')
plt.xlabel('Quarter')
plt.ylabel('Labor Demand Ratio')
plt.xticks(rotation = 45)

sns.despine()
plt.show()
```

```
lag_vars = ['labor_market_stress_index', 'labor_demand_ratio_roll',
'real_wage_growth_rolling',
            'weekly_hours', 'bea_personal_income', 'hps_anxiety_pct',
'hps_depression_pct']
# create 1-quarter lagged versions
for col in lag_vars:
    master_df[col + '_lag1'] = master_df[col].shift(1)
# drop rows with missing lagged data
master_df = master_df.dropna().reset_index(drop = True)
target = 'gt_distress_index'
features = ['labor_market_stress_index_lag1', 'labor_demand_ratio_roll_lag1',
'real_wage_growth_rolling_lag1',
            'weekly_hours_lag1', 'bea_personal_income_lag1']
model_df = master_df[features + [target]].dropna().copy()

# split into predictors and target
X = model_df[features]
y = model_df[target]

# train/test split
split_index = int(len(model_df) * 0.8)
X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]
y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]
# correlation matrix
corr = model_df.corr(numeric_only = True)

# mask upper triangle for cleaner visualization
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure()

sns.heatmap(corr,
            mask = mask,
            cmap = 'coolwarm',
            annot = True,
            fmt = '.2f',
            linewidths = 0.5,
            square = True,
            cbar_kws = {'shrink': 0.8})

plt.title('Feature Correlation Matrix (Pre-Modeling)')
plt.xlabel('')
plt.ylabel('')
plt.xticks(rotation = 45, ha = 'right')
plt.yticks(rotation = 0)
sns.despine(left = True, bottom = True)

plt.show()
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

lr = LinearRegression()

# standardize features for linear regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
```

```
print('Linear Regression Performance')
print('RMSE:', mean_squared_error(y_test, y_pred_lr, squared=False))
print('R2:', r2_score(y_test, y_pred_lr))
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_estimators = 500, random_state = 42)

rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print('Random Forest Performance')
print('RMSE:', mean_squared_error(y_test, y_pred_rf, squared=False))
print('R2:', r2_score(y_test, y_pred_rf))
from xgboost import XGBRegressor

xgb = XGBRegressor(n_estimators = 500,
                   learning_rate = 0.05,
                   max_depth = 4,
                   random_state = 42)

xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)

print('XGBoost Performance')
print('RMSE:', mean_squared_error(y_test, y_pred_xgb, squared=False))
print('R2:', r2_score(y_test, y_pred_xgb))
model_results = {'Model': ['Linear Regression', 'Random Forest', 'XGBoost'],
                 'RMSE': [mean_squared_error(y_test, y_pred_lr, squared = False),
                          mean_squared_error(y_test, y_pred_rf, squared = False),
                          mean_squared_error(y_test, y_pred_xgb, squared = False)],
                 'R2': [r2_score(y_test, y_pred_lr),
                        r2_score(y_test, y_pred_rf),
                        r2_score(y_test, y_pred_xgb)]}

results_df = pd.DataFrame(model_results)
print(results_df)
rf_importance = pd.DataFrame({'feature': features,
                              'importance': rf.feature_importances_
                             }).sort_values('importance', ascending = False)

print(rf_importance)
plt.figure(figsize = (8,5))
plt.barh(rf_importance['feature'], rf_importance['importance'])
plt.gca().invert_yaxis()
plt.title('Random Forest Feature Importance')
plt.xlabel('Importance Score')

plt.show()
xgb_importance = pd.DataFrame({'feature': features, 'importance':
xgb.feature_importances_
                              }).sort_values('importance', ascending = False)

print(xgb_importance)
plt.figure(figsize = (8,5))
plt.barh(xgb_importance['feature'], xgb_importance['importance'])
plt.gca().invert_yaxis()
plt.title('XGBoost Feature Importance')
plt.xlabel('Importance Score')
plt.show()
plt.figure(figsize = (8,5))
plt.scatter(y_test, y_pred_rf)
plt.xlabel('Actual Burnout Index')
plt.ylabel('Predicted Burnout Index')
```

```
plt.title('Random Forest Predictions vs Actual')
plt.show()
# quarter-to-quarter changes in key predictors
master_df['stress_change'] = master_df['labor_market_stress_index'].diff()
master_df['wage_change'] = master_df['real_wage_growth_rolling'].diff()
master_df['demand_change'] = master_df['labor_demand_ratio_roll'].diff()
master_df['hours_change'] = master_df['weekly_hours'].diff()

# lagged changes
master_df['stress_change_lag1'] = master_df['stress_change'].shift(1)
master_df['wage_change_lag1'] = master_df['wage_change'].shift(1)
master_df['demand_change_lag1'] = master_df['demand_change'].shift(1)
master_df['hours_change_lag1'] = master_df['hours_change'].shift(1)

# define burnout change as target
master_df['burnout_change'] = master_df['gt_distress_index'].diff()

target = 'burnout_change'
features = ['stress_change_lag1', 'wage_change_lag1',
            'demand_change_lag1', 'hours_change_lag1']

# random forest configured for change prediction
rf = RandomForestRegressor(n_estimators = 300,
                           max_depth = 3,
                           min_samples_leaf = 2,
                           random_state = 42)
# time-series cross-validation
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
scores = []

for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    rf.fit(X_train, y_train)
    preds = rf.predict(X_test)

    rmse = mean_squared_error(y_test, preds, squared = False)
    scores.append(rmse)

print('Average RMSE:', np.mean(scores))
# re-sort data and create time index
master_df = master_df.sort_values('quarter').reset_index(drop = True)
master_df['time_index'] = range(len(master_df))

# extract quarter number for seasonality
master_df['quarter_num'] = master_df['quarter'].str[-1].astype(int)

features = ['stress_change_lag1', 'wage_change_lag1', 'demand_change_lag1',
            'hours_change_lag1', 'time_index', 'quarter_num']
# rebuild modeling dataset
model_df = master_df[features + [target]].dropna().copy()

X = model_df[features]
y = model_df[target]

rf = RandomForestRegressor(n_estimators = 300,
                           max_depth = 3,
                           min_samples_leaf = 2,
                           random_state = 42)
```

```
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
print(rmse)
plt.figure()
plt.plot(master_df['quarter'],
         master_df['labor_market_stress_index'],
         label = 'Labor Market Stress',
         color = palette['stress'])
plt.plot(master_df['quarter'],
         master_df['gt_distress_index'],
         label = 'Burnout Search Index',
         color = palette['burnout'])
plt.legend()
plt.title('Labor Market Stress vs Workforce Burnout Signals')
plt.xlabel('Quarter')
plt.xticks(rotation = 45)

sns.despine()
plt.show()
plt.figure()
sns.barplot(x = 'importance',
            y = 'feature',
            data = rf_importance,
            color = palette['economic'])
plt.title('Random Forest Feature Importance')
plt.xlabel('Importance Score')
plt.ylabel('Predictor')

sns.despine()
plt.show()
plt.figure()
plt.scatter(y_test,
            y_pred_rf,
            color = palette['prediction'])
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         linestyle = '--',
         color = 'gray')
plt.title('Random Forest Predictions vs Actual Burnout Index')
plt.xlabel('Actual Burnout Index')
plt.ylabel('Predicted Burnout Index')

sns.despine()
plt.show()
corr = model_df.corr(numeric_only = True)
mask = np.triu(np.ones_like(corr, dtype = bool))

plt.figure()
sns.heatmap(corr,
            mask = mask,
            cmap = 'coolwarm',
            annot = True,
            fmt = '.2f',
            linewidths = 0.5,
            square = True,
            cbar_kws = {'shrink': 0.8})
plt.title('Correlation Matrix of Economic Indicators and Burnout Signals')
plt.xticks(rotation = 45, ha = 'right')
plt.yticks(rotation = 0)

sns.despine(left = True, bottom = True)
```

```
plt.show()
```